

```
1  #include <iostream>
2  #include <stdlib.h>
3  #include <cstring>
4  #include "Format.h"
5  #include "Element.hpp"
6  #include "Node.hpp"
7  #include "DataControl.h"
8  #include "InitAndTerm.h"
9  #include "UsageException.h"
10 using namespace std;
11 extern Node<Format> *head, *tail;
12 extern InitAndTerm *cInt;
13
14 DataControl::DataControl()
15 {
16     _pf[0] = &DataControl::selectInsert;
17     _pf[1] = &DataControl::selectDelete;
18     _pf[2] = &DataControl::selectPrint;
19     _pf[3] = &DataControl::selectEnd;
20 }
21 int DataControl::insertNode(Node<Format> *head, Node<Format> *tail, Format data) {
22     Element<Format> *newNode, *list;
23
24     list = head->getNext();
25     while (list != static_cast<Element<Format>*>(tail)) {
26         if ((list->getData()).getNum() == data.getNum() ) {
27             cout << "input fail. same number exist." << endl;
28             return 0;
29         }
30         list = list->getNext();
31     }
32
33     Format _data(data.getNum(), data.getAge(), data.getName());
34     newNode = new Element<Format>(_data);
35
36     newNode->setPrev(tail->getPrev());
37     newNode->setNext(static_cast<Element<Format>*>(tail));
38
39     if (head->getCnt() == 0) {
40         head->setNext(newNode);
41     }
42
43     tail->getPrev()->setNext(newNode);
44     tail->setPrev(newNode);
45
46     head->setCnt(head->getCnt() + 1);
47     tail->setCnt(tail->getCnt() + 1);
48     return 0;
49 }
50 int DataControl::deleteNode(Node<Format> *head, Node<Format> *tail, int num) {
51     Element<Format> *list;
52     list = head->getNext();
53     while (list != static_cast<Element<Format>*>(tail)) {
54         if ((list->getData()).getNum() == num) {
55             if( list->getPrev() == static_cast<Element<Format>*>(head) ){
56                 static_cast<Node<Format>*>( list->getPrev() )->setNext(list->getNext());
57             }else {
58                 list->getPrev()->setNext(list->getNext());
59             }
60             if (list->getNext() == static_cast<Element<Format>*>(tail)) {
61                 static_cast<Node<Format>*>( list->getNext() )->setPrev(list->getPrev());
62             }else {
63                 list->getNext()->setPrev(list->getPrev());
64             }
65             head->setCnt(head->getCnt() - 1);
66             tail->setCnt(tail->getCnt() - 1);
67             delete list;
68             return 0;
69         }
70         list = list->getNext();
71     }
72     cout << "no match number" << endl;
73     return 0;
74 }
75 int DataControl::printNode(Node<Format> *head, Node<Format> *tail) {
76     Element<Format> *list;
77     list = head->getNext();
78     cout << "|num|name|age|" << endl;
79     while (list != static_cast<Element<Format>*>(tail)) {
80         list->getData().showData();
81         list = list->getNext();
82     }
83     return 0;
84 }
85 int DataControl::selectInsert(void) {
86     int _num = 0, _age = 0;
87     char _tempName[NAME_LENGTH] = { '\0', };
88     char c[1] = { '\0', };
89     ERROR_TYPE ERRMSG = STATE_OK;
90     try{
91         Format data(-1, -1, c);
92         cout << "input number: " << endl;
93         scanf("%d", &_num);
94         while (getchar() != '\n');
95         if(_num < 0){
96             ERRMSG = MINUS_NUMBER;
97             throw UsageException(ERRMSG);
98         }
99
100         cout << "input name (limit length under " << NAME_LENGTH - 1 << "): " << endl;
101         fgets(_tempName, sizeof(_tempName), stdin);
102         if (_tempName[strlen(_tempName) - 1] == '\n') {
103             _tempName[strlen(_tempName) - 1] = '\0';
104         }
105         fflush(stdin);
106         if(strlen(_tempName) > NAME_LENGTH){
107             ERRMSG = LENGTH_OVER;
108             throw UsageException(ERRMSG);
109         }
110         cout << "input age: " << endl;
111         scanf("%d", &_age);
112         while (getchar() != '\n');
113         if(_age < 0 ){
114             ERRMSG = MINUS_NUMBER;
115             throw UsageException(ERRMSG);
116         }
117         data.setMember(_num, _age, _tempName);
118         insertNode(head, tail, data);
119     }
120     catch(UsageException& err){
121         err.ShowErrorInfo();
122     }
123     return 0;
124 }
125 int DataControl::selectDelete() {
126     int num;
127     cout << "input number for delete: " << endl;
128     if (scanf("%d", &num) == 0) {
129     }
130     deleteNode(head, tail, num);
131     return 0;
132 }
133 int DataControl::selectPrint(void) {
134     printNode(head, tail);
135     return 0;
136 }
137 int DataControl::selectEnd(void) {
138     cInt->saveData(head, tail);
139     cInt->releaseNode(head, tail);
140     return 0;
141 }
142 int DataControl::exeFunction(int num, int &res)
143 {
144     (this->*_pf[num])();
145     res = 0;
146     return 0;
147 }
148
```